

compute-badges

scripts/compute-badges.sh — §11.4.259 machine-derived README badge regenerator

Revision: 3 **Last modified:** 2026-09-26T14:26:37Z **Status:** active **Item:** BOB-118 (§11.4.6 bluff audit — task-ad3205a9)

Overview

`scripts/compute-badges.sh` regenerates the “python tests” and “frontend tests” shields.io badges in `README.md`, plus the matching Contributing- section bullets and a corroborating `## Test counts` table in `docs/TESTING.md`, from REAL numbers produced by a live invocation of the tools that actually count those tests — never a hand-typed or carried-forward number. It is the fix for BOB-118 and the standing mechanism that keeps the same class of bug from recurring.

Why this exists — BOB-118

`docs/qa/task-ad3205a9-*` (a §11.4.6 no-guessing bluff audit) found that `README.md`’s badge row read `python tests-585 passing` and `frontend tests-182 passing`, while a real `pytest --collect-only over tests/` collected **5248** tests — roughly 9x more than the badge claimed. The cited “authoritative source”, `docs/TESTING.md`, contained **zero** occurrences of either number anywhere in the file — the badge could not be corroborated by any tracked document. This is exactly the class of defect §11.4.259 exists to make structurally impossible: every README badge MUST be machine-derived, never hand-picked.

What it computes

Badge / bullet	Real command	Notes
python tests badge (whole tree)	<code>pytest --collect-only -q tests/ --import-mode=importlib</code>	Parses pytest's own N tests collected summary line.
Contributing bullet “unit + e2e + contract”	<code>sum of pytest --collect-only -q tests/{unit,e2e,contract}/</code>	Scoped to match the bullet's own stated suite scope — never the whole-tree number under a narrower label.
frontend tests badge	<code>frontend/node_modules/.bin/vitest list --run</code>	Exact per-test-case count. Falls back to a labelled grep proxy over <code>it(/test(declarations when node_modules is absent (never silently presented as exact).</code>
## Test counts table in docs/TESTING.md	all of the above, plus <code>tests/integration/</code>	The corroborating source the badge audit found missing.

Two badges (challenges, pre-build invariants) were verified fresh in a prior session and are cross-checked (not rewritten) by this script — see the counts printed to stdout on every run.

Usage

```
# Compute + rewrite README.md and docs/TESTING.md in place
scripts/compute-badges.sh
```

```
# Compute only – print whether the badges are stale, write
  nothing.
# Exit 0 = in sync, exit 2 = stale (advisory; callers must not
  treat
# this as fatal per §11.4.234 always-unblocked).
scripts/compute-badges.sh --check
```

```
# Point at alternate files (used by the paired §1.1 mutation test)
scripts/compute-badges.sh --check --readme /tmp/fixture-README.md
                        --testing-md /tmp/fixture-TESTING.md
```

Wording: “collected” not “passing”

The regenerated badges say “N **collected**”, not “N passing”. Running the full 5248-test tree (many of which need live services, Docker/Podman, or hardware) end-to-end is outside what a badge-refresh script can honestly assert on every invocation. “Collected” is the claim this script can always verify in seconds with zero infrastructure dependencies: the test exists and pytest/vitest can find it. Asserting “passing” without having actually run every one of those tests in the same invocation would just be a differently-shaped version of the same bluff BOB-118 reports.

Edge cases (honest failure mode)

If the python interpreter cannot be resolved (mirrors `ci.sh`’s fallback chain: `$PYTHON` → `.venv/bin/python` → `python3.13` → `python3.12` → `python3`) or `pytest --collect-only` produces no parseable summary line, the badge is emitted as N/A (<reason>) in shields.io’s lightgrey color — never a fabricated number (§11.4.6). Same discipline for the frontend count when neither `vitest` nor any `*.spec.ts` file can be found.

Export of the files it modifies (§11.4.65, BOB-249)

In apply mode the script rewrites `README.md` and `docs/TESTING.md`, then regenerates **only those two files’** `.html/.pdf/.docx` twins via `scripts/generate_markdown_exports.sh <file.md> ...` (explicit-scope mode). A target passed with `--readme / --testing-md` that resolves **outside** the repository root is a test fixture: its export is skipped and the skip is printed. If no in-repo target was modified, the export step is skipped and says so. A failed export stays a loud, non-fatal warning. A symlinked `README/TESTING_MD` is written **through** the link at its real target (never replaced by a regular file), and its export is skipped when that real target lies outside the repository (guard: `tests/unit/test_compute_badges_realpath_scope.sh`).

Before this, the export step ran the generator with **no** argument — a full sweep of root + docs/ + scripts/ of the real tree on every run, including the fixture runs of tests/unit/test_compute_badges_carrier_match.sh and tests/unit/test_compute_badges_script.py. Measured in a scratch copy, one such test run regenerated 42 unrelated tracked twins. Guard: tests/unit/test_compute_badges_export_scope.sh.

Related scripts

- scripts/pre_build_verification.sh invariant CM-BADGE-FRESHNESS-CHECK calls compute-badges.sh --check as a non-blocking WARN (never FAIL, per §11.4.234) so a drifted badge surfaces on every pre-build sweep instead of silently aging for months.
- docs/TESTING.md — the corroborating authoritative source this script keeps in sync.
- scripts/generate_markdown_exports.sh — the exporter, called in explicit-scope mode for the two modified files only.

Last verified

2026-09-26: the export-scope guard passes (fixture run touches no in-tree twin and announces the skip; an in-tree run regenerates only README + TESTING twins).

2026-08-18, this session: --check correctly reported the pre-fix README as stale (5248 vs the stale 585, 371 vs the stale 182), then a clean run regenerated both files and a follow-up --check reported in-sync. See docs/qa/task-bob118-fix/ for the captured before/after evidence.